OpenGL Viewer

Modelview Transformation.

In the previous assignment, I calculated the model view by using the orthographic and perspective matrices as we were shown during class lectures. To use these matrices from class, I had to ensure OpenGL used GL_GREATER with GL_DEPTH_TEST because the Z axis is inverted.

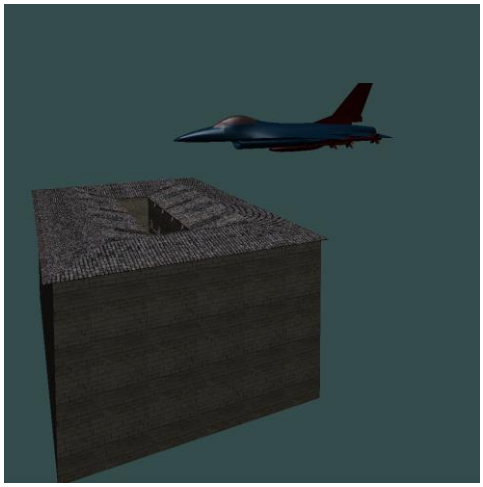
```
orthographic = glm::transpose(glm::mat4(
    2.0f/(r-l), 0.0f, 0.0f, -(r+l)/(r-l),
    0.0f, 2.0f/(t-b), 0.0f, -(t+b)/(t-b),
    0.0f, 0.0f, 2.0f/(n-f), -(n+f)/(n-f),
    0.0f, 0.0f, 0.0f, 1.0f
));

perspective = glm::transpose(glm::mat4(
    n, 0.0f, 0.0f, 0.0f,
    0.0f, n, 0.0f, 0.0f,
    0.0f, 0.0f, n + f, -f * n,
    0.0f, 0.0f, 1.0f, 0.0f
));
```

The orthographic and perspective matrices are multiplied to create the projection matrix that is passed to the shaders along with the view and model matrices.

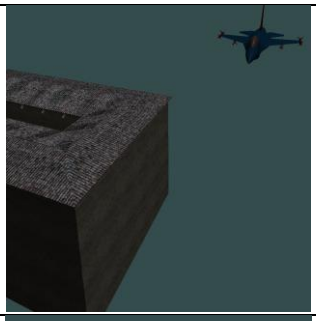
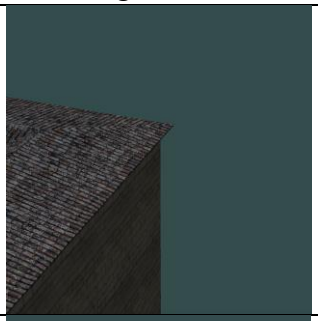
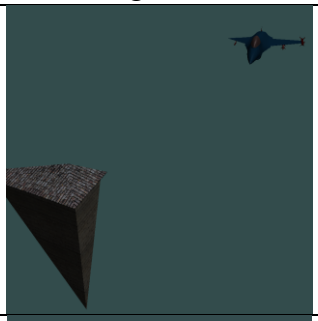
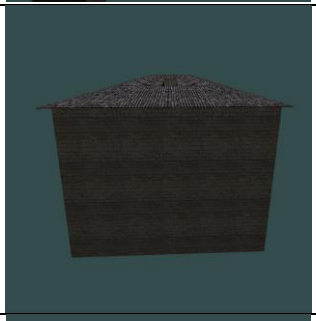
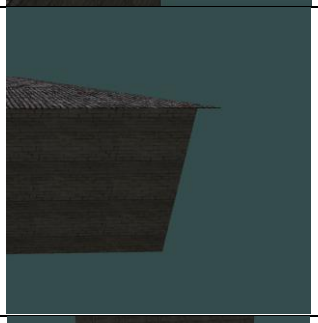
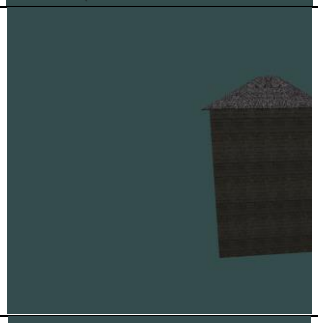
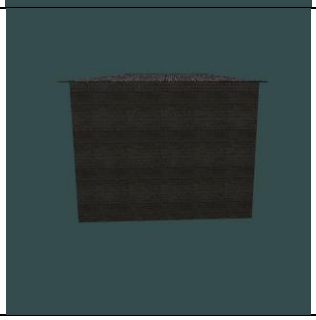
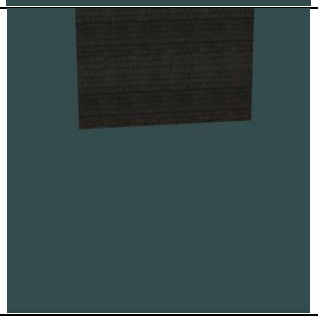
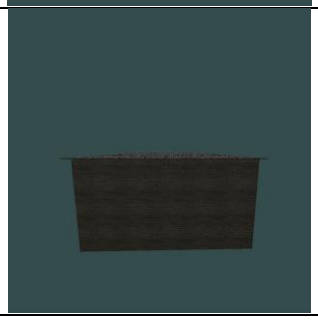


Orthographic



Perspective

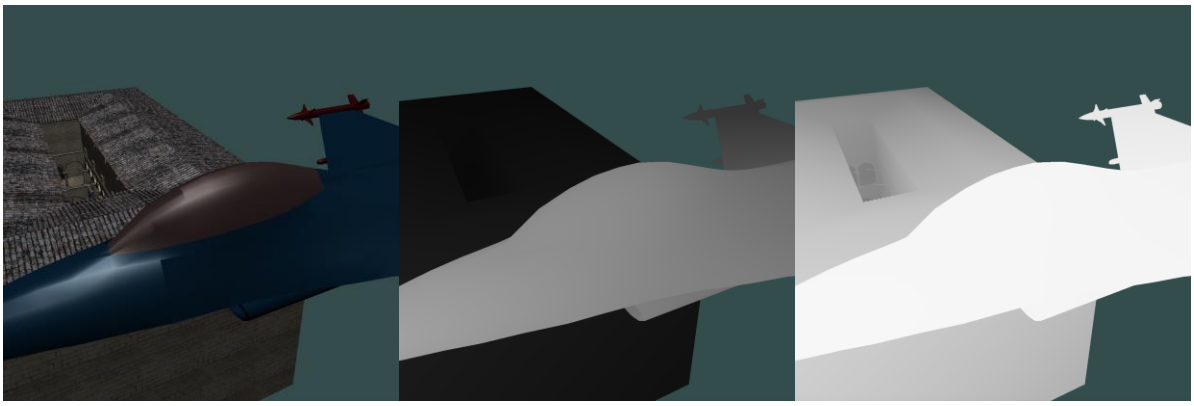
The following table shows the effects of changing the parameters in the perspective matrix.

Parameter	Control	Increasing	Decreasing
Increase: N, Decrease: F			
L (inverted for R)			
T (inverted for B)			

Increasing or decreasing each of the parameters in the projection matrix results in visual differences in the image. Increasing N effectively ‘zooms’ the image by bringing the near plane closer to the objects being rendered. Decreasing F effectively changes the draw distance of the rendering, as we can see clipping with the far plane in the figure above. Changing L and R effectively skew the rendering horizontally, stretching and compressing the image to the left or right. Changing T and B do the same thing, but along the vertical axis.

Z-buffer View

To show the Z-buffer, I added a Z-buffer view condition to the fragment shader. This sets the final color of the pixel to the gl_FragCoord.z value. To show the difference between z', which is shown in the Z-buffer, and the z value, I added a Z view condition, this time to the vertex shader. This sets the color based on the raw z value in camera space (eyePos.z).

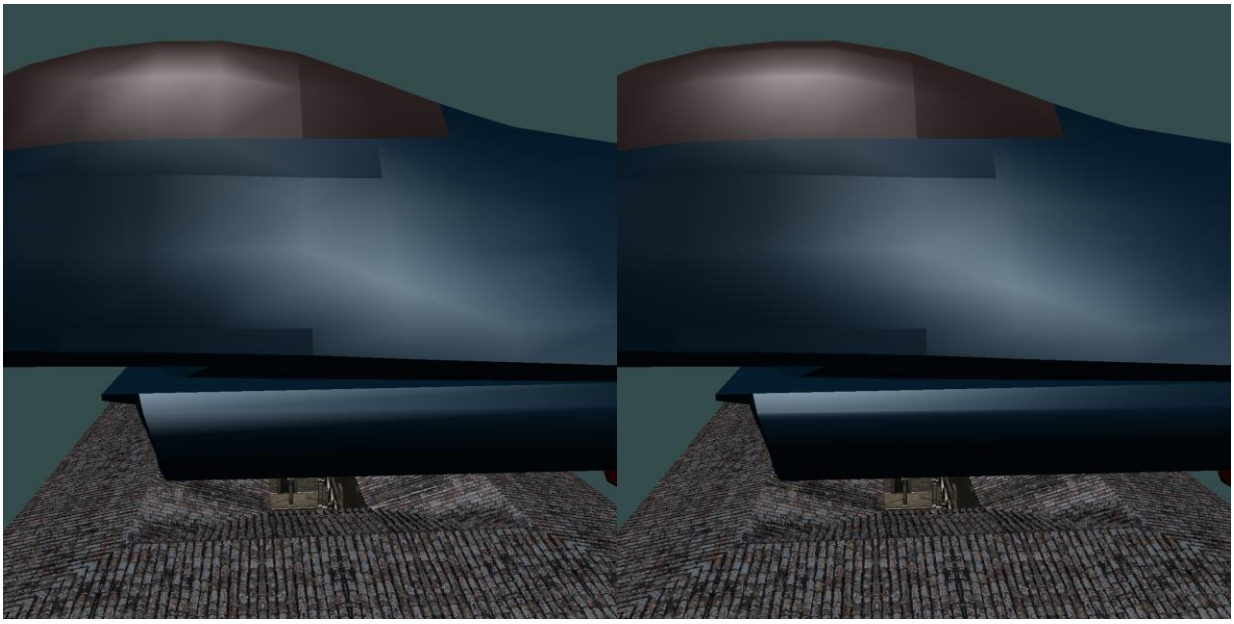


Control Z-Buffer View (z') Z View (z)

From these figures we can see the non-linear nature of the Z-buffer, where the value approaches its maximum very fast. This is because 90% of the values representable in the Z-buffer are within a few virtual meters of the camera. We can see the contrast with the camera space z value which is linear and fades to black much slower.

Gouraud and Phong Shading

To implement these shading techniques, I first needed to pass the light position and parameters for the ambient, diffuse, and specular lighting to the shaders. Gouraud shading is implemented in the vertex shader using the same equations and methods as we did in our Ray Tracer. When the final color is passed from the vertex shader to the fragment shader, the hardware interpolates each pixel between the vertices. This shading method results in a flatter look than Phong as the lighting is only calculated at the vertices. Phong shading on the other hand calculates lighting in the fragment shader per pixel. This creates a smooth look, hiding the edges in the mesh.



Gouraud Shading Phong Shading

Texture Mapping

To implement texture mapping, I rewrote my .obj file parser, giving it the ability to map faces to materials in the corresponding .mtl file. The first thing it does is parse the .mtl file into an unordered map where the material names are keys and its properties such as texture or color if it lacks a texture, are values. Then it parses the .obj file for vertices, vertex normals, and texture coordinates, which are written into the VBO. It then parses face information such as the indexes for the vertex, normal, and texture coordinates into the EBO. For each grouping of faces sharing a material, they are appended to a list of groupings of faces for their respective material. These groupings are represented as a pair where the first value is the offset of the grouping in the EBO and the count of faces in the grouping. This enables us to bind a texture and render every face associated with it before binding the next texture and rendering its faces.

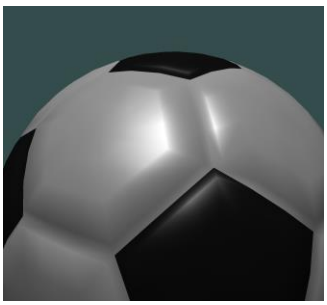
The textures are loaded using stb_image.h and an OpenGL texture object is created. Within the vertex shader, we pass the texture coordinate of the vertex to the fragment shader. In the fragment shader, we sample the texture color at the given UV coordinate, and the pre-lighting color is assigned to it.

With these improvements, we can render large models with multiple textures. The model I chose was Sponza as it is famous in the rendering space, and a .obj file, .mtl file, and textures are readily available for it.

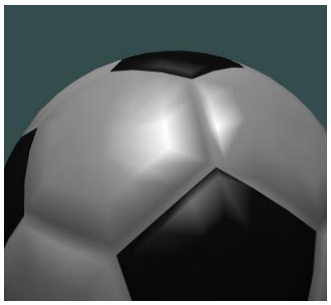


Bonus: Flat Shading

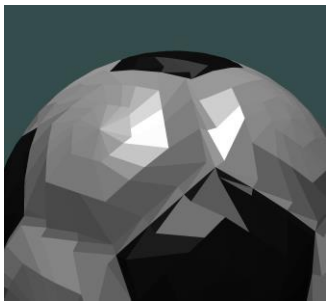
To implement flat shading, I added a flat in and out color and specular variable to the vertex and fragment shaders. It uses the same lighting calculations as before, in the vertex shader like Gouraud. By passing the results through a flat variable it stops the hardware from automatically interpolating values, creating a completely flat shading across polygons.



Phong



Gouraud



Flat